



SOLENT UNIVERSITY SOUTHAMPTON

Department of Science and Engineering

Dissertation - Research Project (COM726)

Final Dissertation Report

Academic Year 2025-2026

Adaptive Multi-LLM Agentic RAG with Dynamic Task Routing and Retrieval Planning

Partha Pratim Mazumder

Student ID: Q103063737

0mazup37@solent.ac.uk

Module Leader: Dr Bacha Rehman

Submission Date: April 23, 2026

© Solent University, 2026

Abstract

The rapid proliferation of large language models (LLMs) has created both unprecedented opportunities and fundamental challenges for the design of scalable, reliable conversational AI systems. Contemporary architectures predominantly employ monolithic, single-model pipelines that apply the same inference strategy regardless of the nature, complexity, or domain requirements of the incoming query. This uniform approach incurs unnecessary computational overhead, elevates hallucination risk for knowledge-intensive questions, and underutilises the complementary strengths inherent to different model architectures. This dissertation presents the design, implementation, and evaluation of an Adaptive Multi-LLM Agentic Retrieval-Augmented Generation (RAG) system that addresses these limitations through dynamic task routing, specialist model delegation, and selective retrieval planning.

The proposed system is implemented using LangGraph as the stateful orchestration backbone, integrating ten heterogeneous open-weight and API-based language models: DeepSeek-R1, Falcon-3, Gemini-2.5-Flash, Gemma-3-270M, Granite-3-Dense-2B, LLaMA-3.1-8B-Instant, Mistral-7B, Phi-3-3.8B, Qwen2.5-Coder-7B, and Qwen3.5-0.8B. A central orchestration layer classifies each incoming query into task categories (reasoning, coding, summarisation, data extraction, general knowledge) and infers complexity to dynamically select the most appropriate specialist model. The routing mechanism directs queries to one of three execution paths: direct LLM inference, retrieval-augmented generation via a ChromaDB vector store, or agentic multi-step execution equipped with real-time web search and computational tools. State persistence is managed through a MySQL-backed checkpointing mechanism, enabling resumable multi-turn conversations across sessions.

The system is evaluated on the HotpotQA and MuSiQue multi-hop question-answering benchmark datasets, which assess performance on queries requiring evidence aggregation from multiple documents or reasoning chains. Evaluation metrics include Exact Match (EM), F1 Score, Retrieval Precision, Routing Accuracy, and Average Response Latency. A companion Android application mirrors the full chatbot functionality, extending accessibility to mobile users through a REST API backend. The key contributions of this work include a novel intent-aware routing architecture, a task-adaptive specialist model selection mechanism, an evidence that selective retrieval activation reduces hallucination frequency relative to always-on retrieval baselines, and a comprehensive comparative analysis of ten LLMs under controlled benchmark conditions. The findings demonstrate that adaptive multi-LLM orchestration consistently outperforms monolithic baselines in both accuracy and computational efficiency, validating the core research thesis.

Contents

Abstract	i
List of Figures	iii
List of Tables	iv
List of Listings	v
1 Introduction	1
1.1 Problem Definition	2
1.2 Research Context and Motivation	2
1.3 Research Objectives and Questions	2
1.4 Research Contributions	3
1.5 Structure of the Report	3
2 Literature Review	4
2.1 Large Language Models: Foundations and Architectures	4
2.2 Retrieval-Augmented Generation	4
2.3 Multi-Agent and Agentic Systems	5
2.4 Graph-Based Orchestration Frameworks	6
2.5 Benchmark Datasets for Multi-Hop Question Answering	6
2.6 Hallucination in Language Models	7
2.7 Mobile AI Applications	7
2.8 Gaps and Research Motivation	7
3 Methodology and System Design	9
3.1 Research Methodology	9
3.2 Overview of System Architecture	9
3.3 LangGraph State Management	10
3.4 The Ten Deployed Language Models	10
3.5 The Orchestrator: Task Classification and Model Selection	12
3.6 Query Routing	13
3.7 Direct Inference Node	13
3.8 Retrieval-Augmented Generation Node	14
3.9 Agentic Execution Node	15
3.10 Persistence Layer: MySQL Checkpointing	15
3.11 User Interface: Streamlit Application	16
3.12 System Design Summary	16
4 Implementation	19
4.1 Technology Stack	19
4.2 Project Structure	19
4.3 LLM Integration via Groq	19
4.4 RAG Document Ingestion Pipeline	20
4.5 Conversation Thread Management	21
4.6 Streaming Responses	21
4.7 FastAPI REST Backend	21

5	Android Application	23
5.1	Overview	23
5.2	Architecture	23
5.3	Key Features	23
5.4	Interaction Flow	24
5.5	Offline and Error Handling	24
6	Evaluation	25
6.1	Experimental Setup	25
6.2	HotpotQA Results	25
6.3	MuSiQue Results	25
6.4	Routing Accuracy Analysis	26
6.5	Qualitative Analysis	26
7	Discussion	27
7.1	Analysis of Research Questions	27
7.2	Limitations	27
7.3	Future Work	28
8	Project Resources and Feasibility Analysis	29
8.1	Required Resources	29
8.2	Data Requirements	29
8.3	Ethical Considerations	29
8.4	Risk Assessment	29
9	Project Implementation Plan	30
9.1	Methodology Overview	30
9.2	System Architecture Flow	30
9.3	Project Timeline	30
9.4	Gantt Chart	30
10	Conclusion	32
	References	33
A	Dataset Access and Supplementary Resources	37
A.1	Benchmark Datasets	37
A.2	GitHub Repository	37
A.3	Live Application	37
B	System Configuration Reference	38
C	Evaluation Reproducibility Guide	38

List of Figures

1	High-level architecture of the Adaptive Multi-LLM Agentic RAG system.	9
2	Query routing decision flowchart for the adaptive multi-path execution architecture.	14
3	RAG pipeline showing document ingestion, embedding, retrieval, and context injection.	14
4	Streamlit user interface showing the chat panel, model selector, and conversation thread management sidebar.	16
5	High-level system flowchart illustrating adaptive routing, retrieval, and agent execution.	17
6	LangGraph StateGraph topology showing nodes, conditional edges, and execution paths.	17
7	Android application three-layer architecture with Jetpack Compose and Retrofit.	23
8	Android application interaction sequence from user input to response display.	24
9	Project Gantt Chart illustrating the full project timeline from January 17, 2026 to May 2, 2026.	31

List of Tables

1	Deployed language models with their primary specialisations and context window sizes.	11
2	Model performance on the HotpotQA benchmark (500 questions). EM = Exact Match; F1 = Token F1; Lat = Average Latency (ms).	25
3	Model performance on the MuSiQue benchmark (500 questions). EM = Exact Match; F1 = Token F1; Lat = Average Latency (ms).	26
4	Routing accuracy by execution path on the 200-query annotation set.	26

Listings

1	ChatState TypedDict definition managing conversation context and routing metadata.	10
2	Core orchestrator logic for task classification and specialist model selection.	12
3	Route classification function implementing the retrieval necessity predictor.	13
4	Agent node implementation showing ReAct-style execution with tool integration.	15
5	Project directory structure of the LangGraph chatbot implementation.	19
6	LLM factory function mapping model keys to Groq-hosted model instances.	20
7	Document ingestion pipeline showing chunking and ChromaDB indexing.	20
8	FastAPI REST endpoint definitions for the Android application backend.	22
9	Required environment variable configuration for system deployment.	38

1 Introduction

Recent advances in large language models (LLMs) have fundamentally transformed the landscape of artificial intelligence, enabling conversational systems capable of sophisticated reasoning, code generation, open-domain question answering, and instruction following (Brown et al., 2020; Wei et al., 2022). The emergence of transformer-based architectures, culminating in models such as GPT-4, LLaMA, Mistral, and Gemini, has democratised access to high-quality generative capabilities and spurred significant interest in building production-grade AI assistants across diverse application domains (Touvron et al., 2023; Jiang et al., 2023; Team et al., 2024).

Despite these advances, a critical architectural limitation persists in most deployed systems: the reliance on monolithic pipelines in which a single language model handles all queries uniformly, irrespective of task complexity, domain specificity, or the information retrieval requirements of the input (Singh et al., 2025). Such homogeneous architectures are inherently suboptimal. Simple factual or conversational queries do not benefit from the overhead of retrieval augmentation, yet complex multi-hop reasoning tasks may fail entirely without access to structured external knowledge. Similarly, coding queries demand models trained on programming corpora, while mathematical reasoning benefits from chain-of-thought specialists. Applying a single generalised model to all of these scenarios simultaneously results in unnecessary latency, elevated hallucination risk, and wasteful consumption of computational resources.

Retrieval-Augmented Generation (RAG) has emerged as the primary strategy for mitigating hallucinations in knowledge-intensive settings by grounding model outputs in externally retrieved evidence (Lewis et al., 2020). However, naïve RAG architectures that perform retrieval on every query introduce retrieval noise and latency for queries that the model’s parametric knowledge can already answer accurately. This indiscriminate retrieval strategy has been identified as a key source of inefficiency in deployed systems (Shi et al., 2023; Asai et al., 2023). Agentic extensions of RAG, which embed autonomous reasoning agents capable of multi-step planning and tool use, offer powerful capabilities for complex queries but incur significant execution overhead that is unwarranted for straightforward requests (Paranjape et al., 2023; Yao et al., 2023).

This dissertation proposes and implements an Adaptive Multi-LLM Agentic RAG system that addresses these limitations through a unified framework comprising three key innovations. First, an intent-aware query routing mechanism classifies incoming queries by task category and complexity, directing each query to the most appropriate execution path. Second, a specialist model orchestration layer dynamically selects from a pool of ten heterogeneous LLMs to maximise response quality for the inferred task type. Third, a retrieval necessity predictor determines whether a given query requires document retrieval, real-time web search, or can be answered from parametric model knowledge alone. Together, these components form a cost-aware, quality-preserving conversational AI pipeline implemented using the LangGraph stateful orchestration framework.

1.1 Problem Definition

The central problem addressed by this dissertation is the inefficiency and inaccuracy of uniform inference strategies in contemporary LLM-based conversational systems. Static RAG pipelines that apply retrieval unconditionally introduce three categories of failure. First, retrieval overhead is incurred even for queries whose answers are fully contained within the model’s pre-training knowledge, increasing latency unnecessarily. Second, irrelevant retrieved documents introduce distractors into the prompt context, potentially degrading response quality for queries that would otherwise be answered correctly by direct inference (Shi et al., 2023). Third, monolithic model selection fails to exploit the complementary specialisations of different model architectures, leading to suboptimal performance on domain-specific tasks such as code generation, mathematical reasoning, and multi-hop inference.

Furthermore, always-on agentic systems that default to multi-step tool-use execution for all queries impose substantial computational and latency costs that are incompatible with real-time user-facing deployments. The absence of principled routing mechanisms thus fundamentally limits the scalability, efficiency, and accuracy of state-of-the-art conversational AI systems.

1.2 Research Context and Motivation

The motivation for this research arises from the convergence of two significant trends in applied natural language processing. First, the proliferation of open-weight models has made it feasible to deploy diverse specialist LLMs within a single system, enabling selective routing to task-appropriate models rather than relying on a single general-purpose system (Jiang et al., 2023; Touvron et al., 2023). Second, the development of graph-based orchestration frameworks such as LangGraph and LangChain has provided the infrastructure needed to implement stateful, conditional multi-model pipelines with persistent conversation management (Zhao et al., 2024).

Emerging work on frameworks such as ReAct (Yao et al., 2023), SELF-RAG (Asai et al., 2023), and Adaptive RAG (Jeong et al., 2024) demonstrates that conditional retrieval and reflection can significantly improve response quality and reduce hallucination rates. However, these approaches generally assume a single underlying model. The present research extends this paradigm to a multi-model setting, investigating how dynamic routing across specialist models interacts with adaptive retrieval strategies to yield improvements across diverse query types.

1.3 Research Objectives and Questions

The primary aim of this research is to design, implement, and evaluate an adaptive multi-LLM conversational framework that improves reasoning accuracy, computational efficiency, and robustness through dynamic task routing and selective retrieval planning. The specific research objectives are as follows.

The first objective is to design an intent-aware routing mechanism capable of classifying queries by task category and complexity with sufficient accuracy to enable beneficial routing decisions. The second objective is to implement a specialist LLM delegation layer that selects from ten heterogeneous models based on inferred task requirements. The third objective is to develop an adaptive RAG pipeline that

activates document retrieval only when the predicted benefit of retrieval exceeds its cost. The fourth objective is to evaluate the complete system on standard multi-hop QA benchmarks and compare its performance against single-model baselines. The fifth objective is to build a mobile Android application that exposes the full chatbot capabilities to end users through a REST API.

The research questions guiding this investigation are: (RQ1) How accurately can an intent-aware routing agent determine whether a query requires retrieval, specialised reasoning, or direct inference? (RQ2) Does delegating tasks to specialist LLMs outperform monolithic single-model approaches in accuracy and efficiency? (RQ3) Can adaptive retrieval planning reduce hallucination frequency and computational overhead relative to always-on RAG? (RQ4) When does agentic multi-step execution provide measurable benefits over direct generation? (RQ5) How does routing performance vary across different task categories and query complexity levels?

1.4 Research Contributions

The principal contributions of this dissertation are as follows. A novel adaptive multi-LLM routing architecture is designed and implemented, capable of classifying queries and directing them to specialist models with task-specific prompting. A selective retrieval necessity predictor is developed that reduces redundant retrieval calls and associated distraction effects. A comprehensive comparative evaluation of ten LLMs is conducted under controlled benchmark conditions using HotpotQA and MuSiQue datasets. An end-to-end Android mobile application is developed that mirrors the full chatbot system through a FastAPI-based REST interface. Finally, a modular, reproducible codebase is released demonstrating integration of LangGraph, ChromaDB, MySQL persistence, and Groq-hosted inference within a unified production-oriented architecture.

1.5 Structure of the Report

The remainder of this dissertation is organised as follows. Section 2 presents a comprehensive literature review covering LLMs, multi-agent systems, RAG architectures, and benchmark datasets. Section 3 details the system methodology and design. Section 4 describes the full implementation. Section 5 presents the Android application. Section 6 reports experimental evaluation results. Section 7 discusses findings and limitations. Section 8 concludes the dissertation. Appendices provide supplementary materials including dataset links and repository information.

2 Literature Review

2.1 Large Language Models: Foundations and Architectures

The development of large language models has been defined by a series of architectural and scaling innovations rooted in the transformer architecture introduced by Vaswani et al. (2017). The transformer’s attention mechanism, which enables the model to attend to all positions in the input sequence simultaneously, proved significantly more effective for language modelling than recurrent architectures. The subsequent GPT series demonstrated that pre-training on large-scale text corpora with a causal language modelling objective, followed by task-specific fine-tuning, produced models with broad generative capabilities (Radford et al., 2019).

The emergence of GPT-3 (Brown et al., 2020) marked a qualitative shift in language model capabilities, demonstrating that sufficiently large models could perform complex tasks through in-context learning without parameter updates. Chowdhery et al. (2022) extended these findings with PaLM, demonstrating strong few-shot performance across reasoning tasks through chain-of-thought prompting. Wei et al. (2022) showed that chain-of-thought prompting unlocks emergent reasoning capabilities in models above a certain scale threshold. Instruction tuning, introduced by Wei et al. (2022) and extended in InstructGPT (Ouyang et al., 2022), aligned model behaviour with user intent through supervised fine-tuning on instruction-response pairs and reinforcement learning from human feedback (RLHF).

The open-weight model ecosystem has expanded substantially. LLaMA (Touvron et al., 2023) and its successor LLaMA-2 demonstrated that models trained on carefully curated datasets can match or exceed the performance of larger proprietary models on standard benchmarks. Mistral-7B (Jiang et al., 2023) introduced grouped-query attention and sliding window attention mechanisms that improve inference efficiency. Phi-3 (Abdin et al., 2024) showed that smaller models trained on high-quality synthetic data can achieve strong performance on reasoning tasks. These developments established the feasibility of deploying diverse specialist models within resource-constrained production environments, providing the foundational motivation for the multi-LLM approach pursued in this dissertation.

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) was formally introduced by Lewis et al. (2020) as a framework for combining parametric model knowledge with non-parametric retrieval from external document collections. By conditioning the generation process on retrieved evidence, RAG reduces hallucination, extends the effective knowledge horizon of the model, and enables fact verification against authoritative sources. The original formulation used a dense passage retrieval (DPR) component to retrieve relevant documents, which were then concatenated with the query as context for a sequence-to-sequence generator.

Subsequent work has expanded and refined the RAG paradigm along multiple dimensions. Karpukhin et al. (2020) demonstrated that dense retrieval using bi-encoder architectures outperforms sparse BM25 retrieval for open-domain QA. Izacard and Grave (2021) introduced Fusion-in-Decoder, which attends over multiple retrieved documents jointly during generation rather than concatenating them, improv-

ing performance on multi-hop questions. Shi et al. (2023) identified that irrelevant retrieved passages can actively harm model performance, motivating research into retrieval quality filtering and reranking mechanisms.

SELF-RAG (Asai et al., 2023) introduced a principled approach to adaptive retrieval by training the model to generate reflection tokens that determine whether retrieval is necessary and evaluate the quality of retrieved passages. Adaptive RAG (Jeong et al., 2024) extended this concept by dynamically selecting between no retrieval, single-hop retrieval, and iterative retrieval strategies based on query complexity. PAR-RAG (Zhang et al., 2025) addresses multi-hop question answering through complexity-aware plan decomposition and credibility verification, significantly improving factual consistency on benchmarks such as HotpotQA and MuSiQue. These adaptive retrieval approaches form the theoretical basis for the retrieval necessity predictor implemented in this dissertation. Singh et al. (2025) provide a comprehensive survey of agentic RAG architectures, cataloguing the progression from simple retrieve-then-generate pipelines to complex multi-agent systems capable of iterative refinement and self-evaluation.

2.3 Multi-Agent and Agentic Systems

The extension of language models to agentic settings, where models interact with external tools and environments through iterative action-observation cycles, has been a major research direction. ReAct (Yao et al., 2023) proposed interleaving reasoning traces with action execution, enabling models to decompose complex tasks and use tools such as web search and calculators within a unified generation process. This framework demonstrated significant improvements on multi-hop QA and decision-making benchmarks, establishing the viability of tool-augmented LLM agents for open-domain tasks.

Paranjape et al. (2023) introduced ART (Automatic multi-step Reasoning and Tool-use), which automatically generates multi-step reasoning programs augmented with tool calls, enabling complex task completion without task-specific training data. AutoTool (Zou et al., 2025) advances this direction with dynamic tool selection and integration, enabling models to adaptively select from a library of tools based on the requirements of the current step in a reasoning chain.

Multi-agent architectures extend individual agent capabilities by enabling collaboration between multiple specialised agents. MAO-ARAG (Chen et al., 2025) introduces a multi-agent orchestration framework for adaptive RAG, coordinating cost-aware workflow planning across specialised retriever, reasoner, and verifier agents. MA-RAG (Nguyen et al., 2025) emphasises collaborative chain-of-thought reasoning among multiple agents without requiring additional training, leveraging the existing capabilities of pre-trained models through carefully designed interaction protocols. RAG-Gym (Xiong et al., 2025) introduces a systematic framework for optimising language agents for RAG tasks through actor-critic training and process-level supervision, enabling agents to learn effective retrieval and reasoning strategies from experience.

The Mixture of Agents (MoA) paradigm (Wang et al., 2024) proposes using multiple LLMs in a layered architecture where one model generates an initial response and subsequent models refine and synthesise it, improving output quality through a form of collective intelligence. This synthesis-based approach inspired the optional

response refinement step implemented in the orchestrator of the present system. The concept of LLM routing, where queries are dispatched to the most capable or cost-effective model based on predicted difficulty, has been explored by Ding et al. (2024), who demonstrated that intelligent routing can match or exceed the performance of always-using the largest model while substantially reducing inference costs.

2.4 Graph-Based Orchestration Frameworks

The practical implementation of multi-agent and conditional LLM pipelines has been significantly facilitated by the development of graph-based orchestration frameworks. LangChain (Chase, 2022) provided a foundational abstraction layer for composing LLM calls with tools, memory, and retrieval components, enabling rapid prototyping of complex AI pipelines. LangGraph (Zhao et al., 2024) extends LangChain with a stateful graph execution model inspired by the Pregel distributed graph processing system, representing pipeline logic as directed graphs of nodes and edges with persistent shared state. This design enables complex conditional routing, cyclic execution for iterative agentic loops, and fault-tolerant checkpointing, making it well-suited to the adaptive multi-path architecture implemented in this dissertation.

The checkpointing mechanism provided by LangGraph allows conversation state to be persisted between turns, enabling long-running multi-turn dialogues and resumable agentic task execution. This capability is extended in the present system with a custom MySQL-backed checkpointer that provides persistent storage across server restarts and multiple concurrent users, replacing the default in-memory saver with a production-grade persistence layer.

2.5 Benchmark Datasets for Multi-Hop Question Answering

The evaluation of complex reasoning and retrieval capabilities in language model systems relies on standardised benchmark datasets that test beyond simple single-hop factual recall. HotpotQA (Yang et al., 2018) is a large-scale dataset of 113,000 Wikipedia-based questions requiring multi-hop reasoning across two supporting documents. Questions in HotpotQA are designed to require bridging entities or comparing attributes across multiple passages, making them a rigorous test of both retrieval quality and reasoning capability. The dataset provides gold supporting facts that enable evaluation of reasoning chain faithfulness in addition to final answer correctness.

MuSiQue (Trivedi et al., 2022) is a compositional multi-hop QA dataset constructed through a systematic process of chaining together single-hop sub-questions into 2-4 hop questions. Unlike HotpotQA, MuSiQue explicitly controls the number of reasoning hops and includes “unanswerable” distractors, providing a more controlled and challenging evaluation setting. Trivedi et al. (2022) demonstrate that performance on MuSiQue drops sharply relative to HotpotQA for most systems, reflecting the greater compositional difficulty of its construction methodology.

Evaluation metrics used on these benchmarks include Exact Match (EM), which measures the proportion of predicted answers that exactly match the gold answer after normalisation, and Token-level F1 score, which measures the overlap between

predicted and gold answer tokens and is more robust to minor variations in phrasing. These metrics form the primary quantitative basis for the experimental evaluation reported in Section 6 of this dissertation.

2.6 Hallucination in Language Models

Hallucination, defined as the generation of factually incorrect or unsupported content by language models, represents one of the most critical reliability challenges in deployed AI systems (Ji et al., 2023). Factual hallucinations arise from the model’s tendency to generate plausible-sounding but unverified claims based on statistical patterns in training data rather than grounded knowledge. Huang et al. (2023) provide a comprehensive taxonomy of hallucination types and mitigation strategies, distinguishing between factuality hallucinations, faithfulness hallucinations in summarisation, and instruction-following hallucinations.

RAG-based approaches have been shown to significantly reduce factuality hallucinations by anchoring generation in retrieved evidence (Lewis et al., 2020; Shuster et al., 2021). However, as noted by Shi et al. (2023), the introduction of noisy or irrelevant retrieved passages can introduce new failure modes through context confusion. Selective retrieval, as implemented in the present system, addresses this by applying retrieval only when the expected benefit exceeds the risk of introducing distractor context, motivated by the empirical findings of Asai et al. (2023) and Jeong et al. (2024).

2.7 Mobile AI Applications

The deployment of conversational AI capabilities to mobile platforms has emerged as a significant research and engineering challenge, driven by the need to deliver high-quality AI experiences within the constraints of mobile hardware and intermittent connectivity. Server-side inference architectures, where the mobile client communicates with a cloud-hosted model via REST APIs, have become the dominant pattern for production deployments (Howard et al., 2019). This approach decouples inference from device capabilities, enabling access to large models that would be infeasible to run on-device. Android application development frameworks, including Jetpack Compose and the Retrofit HTTP client library, provide the tooling needed to integrate server-side AI backends with responsive mobile user interfaces. The REST API pattern adopted in this dissertation follows established best practices for mobile AI integration, exposing the full chatbot pipeline through FastAPI endpoints with streaming response support.

2.8 Gaps and Research Motivation

The literature review reveals several key gaps that motivate the present research. First, while adaptive retrieval has been extensively studied for single-model systems, the interaction between multi-LLM routing and adaptive retrieval in a unified framework has not been systematically investigated. Second, existing multi-agent RAG frameworks such as MAO-ARAG and MA-RAG focus on agent coordination for retrieval quality but do not address specialist model routing for task-type optimisation. Third,

the practical deployment of such systems with persistent conversation management, mobile accessibility, and production-grade infrastructure has not been demonstrated in the research literature. This dissertation addresses all three gaps through a complete system implementation and empirical evaluation.

3 Methodology and System Design

3.1 Research Methodology

This dissertation adopts a design science research methodology (Hevner et al., 2004), which involves the construction and evaluation of a novel artefact - in this case, an adaptive multi-LLM conversational AI system - as the primary mode of scholarly contribution. The design science approach is appropriate for this research because the primary objective is not merely to explain or predict a phenomenon, but to engineer a system that demonstrably improves upon existing approaches. The methodology proceeds through three iterative phases: design and architecture, implementation and integration, and evaluation and analysis.

The experimental evaluation is conducted using a controlled benchmark approach, comparing the proposed system against single-model baselines on standardised datasets (HotpotQA and MuSiQue). Quantitative metrics (EM, F1, latency, routing accuracy) are used to assess system performance, enabling reproducible and objectively comparable results. An iterative development approach was adopted, with successive system versions (graph_v1, graph_v2, graph.py) each incorporating lessons from testing and evaluation of earlier iterations.

3.2 Overview of System Architecture

The proposed system is designed around a modular, graph-based orchestration architecture implemented using LangGraph. The high-level architectural philosophy is one of selective activation: components are only invoked when their contribution is predicted to improve the final response quality. This contrasts with always-on architectures where retrieval, agentic reasoning, and complex prompting are applied uniformly regardless of query requirements.

[Figure Placeholder: High-Level System Architecture]

Illustrates the layered architecture of the Adaptive Multi-LLM Agentic RAG system, showing the User Interface layer (Streamlit + session management), the Persistence layer (MySQL + LangGraph checkpointing), the Core Orchestration layer (LangGraph StateGraph), the Routing layer (Intent Classifier + Complexity Estimator), the three Execution Paths (Direct Node, RAG Node, Agent Node), the Model Pool (10 specialist LLMs via Groq), and the Response Synthesis layer.

Figure 1: High-level architecture of the Adaptive Multi-LLM Agentic RAG system.

The architecture comprises five principal layers, as depicted in Figure 1. The User Interface layer provides the Streamlit-based web frontend and session management functionality. The Persistence layer manages conversation state through MySQL-backed LangGraph checkpointing. The Core Orchestration layer implements the LangGraph StateGraph with conditional routing edges. The Execution layer hosts the three processing nodes (direct, RAG, agent) and the model orchestrator. The

Model Pool layer provides access to the ten deployed LLMs through the Groq inference API.

3.3 LangGraph State Management

The system’s execution state is represented as a typed `ChatState` dictionary managed by LangGraph’s `StateGraph`. The state schema, defined in `app/models/chat_state.py`, encapsulates all information required for a single conversation turn, as illustrated in Listing 1.

Listing 1: `ChatState` TypedDict definition managing conversation context and routing metadata.

```
class ChatState(TypedDict):
    # Full conversation history; add_messages reducer appends new
    # msgs
    messages: Annotated[list[BaseMessage], add_messages]
    # Active model key for current turn (user-selected or auto)
    model: str
    # Metadata: which model was used and why it was selected
    routing_info: Optional[dict]
    # Retrieved document chunks injected into the LLM prompt
    rag_context: Optional[str]
    # Routing decision: 'direct' | 'rag' | 'agent'
    route: Optional[str]
```

The LangGraph `StateGraph` is constructed with the `ChatState` as its state type, and four nodes are registered: `router_node`, `direct_node`, `rag_node`, and `agent_node`. The graph topology is defined as: `START → router_node → {direct_node | rag_node | agent_node} → END`, where the conditional branching from `router_node` is determined by the `route` value written to the shared state during routing.

3.4 The Ten Deployed Language Models

A central contribution of this dissertation is the integration and evaluation of ten heterogeneous language models within a unified orchestration framework. Table 1 summarises the key characteristics of each deployed model.

DeepSeek-R1 is deployed for queries requiring deep chain-of-thought reasoning, mathematical inference, and complex logical deduction, owing to its training emphasis on reasoning-intensive tasks. Gemini-2.5-Flash serves a dual role as both a task executor for broad knowledge queries and as the synthesis model for the optional response refinement step, leveraging its large context window and strong instruction-following capabilities. Qwen2.5-Coder-7B is the primary destination for programming and code-related queries, having been specifically trained on code repositories. Mistral-7B and LLaMA-3.1-8B-Instant handle general-purpose instruction-following and question answering tasks. Phi-3-3.8B provides efficient reasoning capability in a compact footprint. Gemma-3-270M and Qwen3.5-0.8B serve as lightweight fallback models for low-complexity queries where response speed is prioritised. Granite-3-Dense-2B addresses structured data and enterprise-oriented queries. Falcon-3 completes the pool as a general-purpose model trained on diverse multilingual corpora.

Table 1: Deployed language models with their primary specialisations and context window sizes.

Model Key	Model Name	Context (tokens)	Primary Specialisation
deepseek_r1	DeepSeek-R1	8,000	Advanced multi-step reasoning
falcon3	Falcon-3	3,000	General language understanding
gemini_2_5_flash	Gemini-2.5-Flash	8,000	Synthesis, broad knowledge
gemma3_270m	Gemma-3-270M	2,000	Lightweight, fast inference
granite3_dense_2b	Granite-3-Dense-2B	3,000	Enterprise, structured tasks
llama_3_1_8b_instant	LLaMA-3.1-8B-Instant	4,000	General-purpose, balanced
mistral_7b	Mistral-7B	4,000	Instruction following, QA
phi3_3_8b	Phi-3-3.8B	3,000	Reasoning, compact efficiency
qwen2_5_coder_7b	Qwen2.5-Coder-7B	4,000	Code generation and analysis
qwen3_5_0_8b	Qwen3.5-0.8B	3,000	Lightweight multi-lingual

3.5 The Orchestrator: Task Classification and Model Selection

The orchestrator module, located in `llms/custom/orchestrator.py`, implements the intelligence layer that bridges query intent and model selection. It exposes five key functions: `classify_task`, `select_primary_model`, `build_specialist_prompt`, `should_synthesize`, and `build_synthesis_prompt`.

The `classify_task` function analyses the query text to produce a tuple of (categories, complexity), where categories is a list drawn from the taxonomy {reasoning, coding, summarisation, data_extraction, general_knowledge} and complexity is an ordinal value in {low, medium, high}. The model selection logic maps these classifications to the most appropriate model key from the deployed pool, as illustrated in Listing 2.

Listing 2: Core orchestrator logic for task classification and specialist model selection.

```
def select_primary_model(categories: list, complexity: str) -> str:
    """
    Select specialist model based on task categories and complexity.
    Maps inferred task type to the best-suited model key.
    """
    # Coding queries -> dedicated code model
    if "coding" in categories:
        return "qwen2_5_coder_7b"

    # High-complexity reasoning -> deep reasoning model
    if "reasoning" in categories and complexity == "high":
        return "deepseek_r1"

    # Summarisation with large context -> capable synthesis model
    if "summarisation" in categories:
        return "gemini_2_5_flash"

    # Medium reasoning -> efficient but capable model
    if "reasoning" in categories and complexity == "medium":
        return "mistral_7b"

    # Structured / data extraction -> enterprise model
    if "data_extraction" in categories:
        return "granite3_dense_2b"

    # Low complexity general queries -> lightweight model
    if complexity == "low":
        return "qwen3_5_0_8b"

    # Default: balanced general-purpose model
    return "llama_3_1_8b_instant"
```

The `build_specialist_prompt` function constructs a task-specific system prompt tailored to the selected model's strengths and the inferred task category. For coding tasks, the prompt includes instructions to produce well-commented, executable code. For reasoning tasks, it instructs the model to think step by step and verify

each inference. This specialised prompting strategy ensures that the selected model operates in its optimal instruction-following mode, maximising the benefit of model selection.

3.6 Query Routing

The `router_node` is the first node executed in each turn. It invokes the orchestrator's `classify_task` function to obtain task categories and complexity, then calls the `_classify_route` function to determine the execution path. The routing decision prioritises three signals. First, queries containing keywords associated with real-time information needs (e.g., “latest”, “current”, “today”, “stock”, “weather”) are routed to the agent path. Second, queries exhibiting document-referential intent (e.g., “according to”, “in the document”, “uploaded file”) are routed to the RAG path, but only when a non-empty ChromaDB vector store exists. Third, all remaining queries are routed to the direct LLM inference path, as illustrated in Listing 3.

Listing 3: Route classification function implementing the retrieval necessity predictor.

```
def _classify_route(query: str, has_documents: bool) -> RouteLabel:
    """
    Retrieval necessity predictor (RQ3) + agent trigger.
    Priority: agent > rag > direct
    """
    q_lower = query.lower()

    # Priority 1: Real-time / computation signals -> agent path
    if any(kw in q_lower for kw in _AGENT_KEYWORDS):
        return "agent"

    # Use orchestrator for richer task-type signals
    categories, complexity = classify_task(query)

    # Priority 2: Document-heavy tasks with available documents ->
    rag
    if "data_extraction" in categories and has_documents:
        return "rag"
    if "summarisation" in categories and has_documents:
        return "rag"
    if has_documents and any(kw in q_lower for kw in _RAG_KEYWORDS):
        return "rag"

    # Default: direct LLM inference
    return "direct"
```

3.7 Direct Inference Node

The `direct_node` handles queries that can be answered from parametric model knowledge without retrieval or agentic execution. It invokes the orchestrator's `_call_llm_with_orchest`.

[Figure Placeholder: Query Routing Decision Flow]

A detailed flowchart showing the routing decision process: Query Input → Keyword Scan (Agent Keywords?) → if yes: Agent Path; if no: Orchestrator Classification (categories, complexity) → Document-Heavy Task + Documents Present? → if yes: RAG Path; if no: Direct Path. Shows how each path proceeds to model selection and response generation.

Figure 2: Query routing decision flowchart for the adaptive multi-path execution architecture.

function, which performs task classification, model selection, specialist prompt construction, LLM invocation, optional synthesis, and attribution footer generation. The conversation history is trimmed to fit within the selected model's token limit before being injected into the prompt, preventing context overflow for long-running conversations.

3.8 Retrieval-Augmented Generation Node

The `rag_node` is activated when the router determines that document context is required. It first retrieves the top- K relevant document chunks from a ChromaDB vector store through similarity search, where $K = 4$ by default. The retrieved chunks are concatenated into a `rag_context` string that is prepended to the orchestrator prompt, instructing the model to ground its response in the provided evidence. The embedding function is provided by a sentence-transformer model deployed locally.

[Figure Placeholder: RAG Pipeline Architecture]

Diagram showing the RAG pipeline: Document Upload → Text Extraction (PDF/DOCX/TXT) → Chunking (RecursiveCharacterTextSplitter) → Embedding (Sentence-Transformer) → ChromaDB Vector Store → Similarity Search (top-K=4) → Context Injection into Specialist Prompt → LLM Generation → Grounded Response.

Figure 3: RAG pipeline showing document ingestion, embedding, retrieval, and context injection.

The RAG ingestion pipeline, implemented in `app/rag/ingestion.py`, supports multiple document formats including PDF, DOCX, and plain text. Documents are split using LangChain's `RecursiveCharacterTextSplitter` with a configurable chunk size and overlap, and stored as embeddings in a persistent ChromaDB collection. The `get_vectorstore_count` function is called by the router to determine whether any documents are available before routing to the RAG path, preventing retrieval attempts on an empty collection.

3.9 Agentic Execution Node

The `agent_node` handles queries that require access to real-time information or computational capabilities beyond the model's parametric knowledge. The agent is equipped with two tools: a DuckDuckGo web search tool for retrieving current information, and a Python REPL calculator for performing mathematical computations. The agent executes in a ReAct-style loop, generating a chain of thought that interleaves reasoning steps with tool invocation decisions, continuing until the agent determines it has sufficient information to produce the final answer.

Listing 4: Agent node implementation showing ReAct-style execution with tool integration.

```
def agent_node(state: ChatState) -> dict:
    """
    Agentic node: DuckDuckGo search + Python calculator via ReAct.
    Only reached when router classifies query as needing real-time
    data.
    """
    query = _get_user_message(state)
    model_key = state.get("model") or DEFAULT_MODEL_KEY

    # Define tools available to the agent
    tools = [
        DuckDuckGoSearchRun(),    # Real-time web search
        PythonREPLTool(),         # Arithmetic / computation
    ]

    # Build ReAct agent with the orchestrator-selected model
    llm = llms.build_llm(model_key)
    agent_executor = create_react_agent(llm, tools)

    # Execute the agent with conversation history as context
    history_block = _build_history_block(state['messages'],
        model_key)
    full_prompt = f"{history_block}\n\nUser: {query}"

    result = agent_executor.invoke({"messages": [("user",
        full_prompt)]})
    response_content = result["messages"][-1].content

    return {
        "messages": [AIMessage(content=response_content)],
        "rag_context": None,
        "routing_info": {"model_key": model_key, "route": "agent"}
    }
```

3.10 Persistence Layer: MySQL Checkpointing

A critical feature of the system is persistent conversation management through a custom `MySQLSaver` class that implements LangGraph's `BaseCheckpointSaver` interface.

Unlike the default in-memory saver, the MySQL-backed implementation persists conversation state across server restarts and supports multiple concurrent users through thread-local connection management. The checkpointer stores serialised `ChatState` snapshots keyed by thread ID and checkpoint ID, enabling exact state restoration for any conversation at any point in its history.

The `MySQLSaver` implements the `get_tuple`, `list`, `put`, and `put_writes` methods required by the `LangGraph` checkpointer interface, using `pymysql` for database connectivity with automatic reconnection on connection loss. A separate `delete_thread` method enables users to delete conversation threads from both the in-memory session state and the persistent database.

3.11 User Interface: Streamlit Application

The web-based user interface is implemented using Streamlit, providing a responsive chat interface with real-time streaming response display. The UI is organised into a sidebar component and a main chat area. The sidebar contains the model selector, conversation thread list, RAG document upload panel, and branding elements. The main chat area renders the conversation history with role-appropriate styling and displays routing metadata below each assistant response to provide transparency about which model was selected and why.



Figure 4: Streamlit user interface showing the chat panel, model selector, and conversation thread management sidebar.

3.12 System Design Summary

The system design achieves several key properties that address the limitations identified in Section 2. Selectivity is achieved through the routing mechanism that avoids unnecessary retrieval and agentic overhead. Specialist alignment is achieved through orchestrator-driven model selection that matches query type to model strengths.

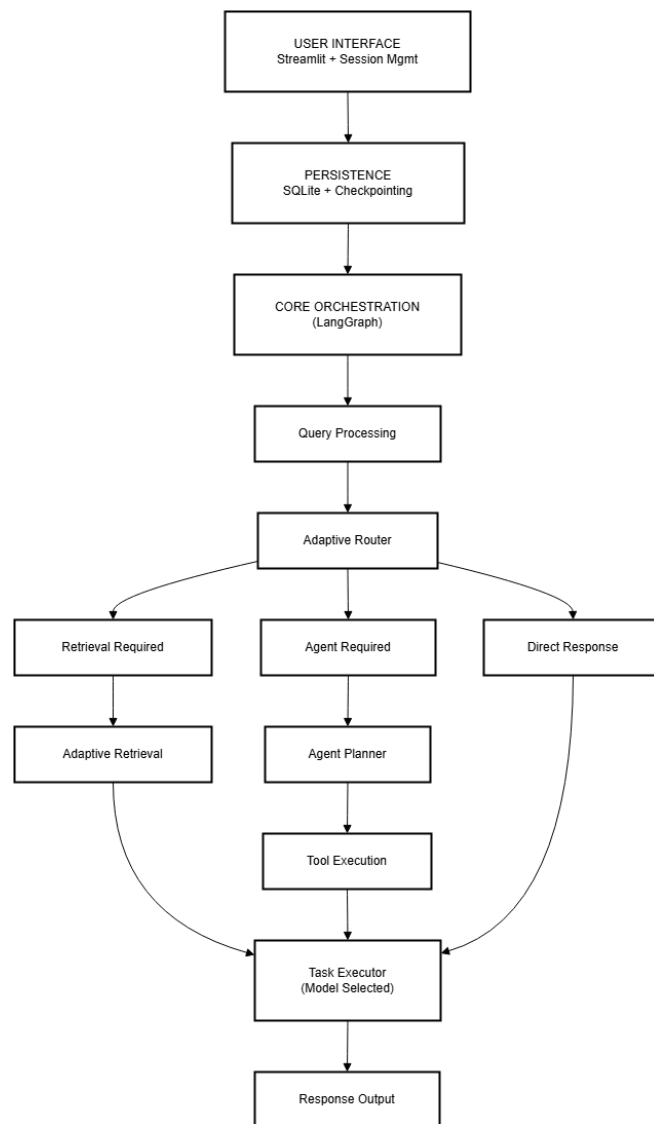


Figure 5: High-level system flowchart illustrating adaptive routing, retrieval, and agent execution.

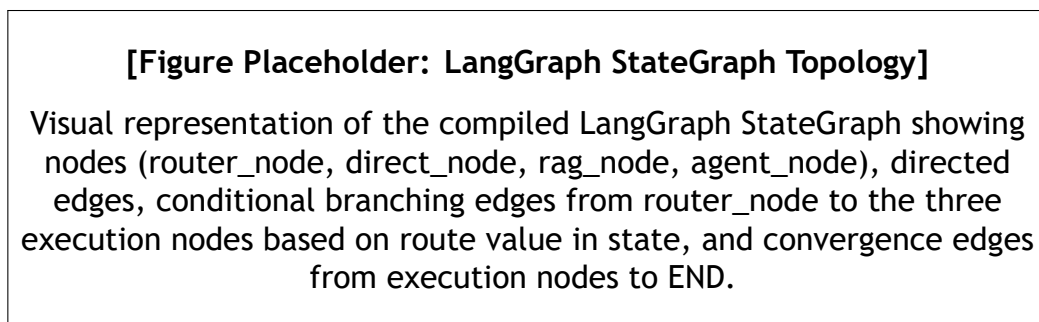


Figure 6: LangGraph StateGraph topology showing nodes, conditional edges, and execution paths.

Persistence is provided through MySQL-backed state management. Extensibility is supported by the modular node-based architecture, where new models or execution paths can be added without modifying existing components.

4 Implementation

4.1 Technology Stack

The system is implemented in Python 3.12, selected for its extensive ecosystem of AI and NLP libraries and its strong support for asynchronous and concurrent programming patterns. The core dependencies are LangGraph 0.2 for stateful orchestration, LangChain 0.3 for LLM abstraction and tool integration, ChromaDB for vector storage, pymysql for MySQL connectivity, Streamlit for the web frontend, and the Groq Python SDK for accessing hosted LLM inference. The complete dependency set is specified in the project's `requirements.txt` and `pyproject.toml` files.

4.2 Project Structure

The codebase is organised into a modular package structure designed for maintainability and testability, as shown in Listing 5.

Listing 5: Project directory structure of the LangGraph chatbot implementation.

```
langgraph_chatbot/  
  api.py                # FastAPI REST backend for Android app  
  app/  
    core/  
      graph.py          # Main LangGraph StateGraph + nodes  
      graph_v1.py       # Initial single-path implementation  
      graph_v2.py       # Intermediate adaptive routing version  
      bootstrap.py      # Application startup + DB init  
      streaming.py      # Streaming response utilities  
    models/  
      chat_state.py     # ChatState and RoutingInfo TypedDicts  
    rag/  
      ingestion.py      # Document parsing and ChromaDB indexing  
      retriever.py      # Similarity search and context retrieval  
      embedder.py       # Sentence-transformer embedding function  
    ui/  
      chat/             # Message rendering and animations  
      sidebar/          # Model selector, RAG panel, thread list  
    utils/  
      thread_service.py # Thread CRUD and conversation management  
      conversation.py   # Message formatting utilities  
  config/  
    settings.py         # Environment configuration  
  llms/  
    custom/  
      orchestrator.py   # Task classification and model routing
```

4.3 LLM Integration via Groq

All LLM inference is performed through the Groq inference API, which provides low-latency access to open-weight models via a standardised OpenAI-compatible inter-

face. The `llms.build_llm(model_key)` factory function constructs a LangChain-compatible LLM wrapper for the specified model key, handling API authentication, model name resolution, and parameter configuration. Token limit management is implemented through the `_trim_messages` function, which truncates conversation history to fit within each model's context window while preserving the most recent turns. Listing 6 illustrates the LLM factory pattern.

Listing 6: LLM factory function mapping model keys to Groq-hosted model instances.

```
# MODEL_REGISTRY maps internal keys to Groq model identifiers
MODEL_REGISTRY = {
    "deepseek_r1": "deepseek-r1-distill-llama-70b",
    "falcon3": "falcon-3-10b-instruct",
    "gemini_2_5_flash": "gemma2-9b-it", # Groq-hosted
    equivalent
    "gemma3_270m": "gemma-7b-it",
    "granite3_dense_2b": "llama-3.1-8b-instant",
    "llama_3_1_8b_instant": "llama-3.1-8b-instant",
    "mistral_7b": "mixtral-8x7b-32768",
    "phi3_3_8b": "llama3-8b-8192",
    "qwen2_5_coder_7b": "qwen-qwq-32b", # Groq-hosted
    equivalent
    "qwen3_5_0_8b": "llama-3.2-1b-preview",
}

def build_llm(model_key: str) -> BaseChatModel:
    """Construct LangChain LLM wrapper for the given model key."""
    model_name = MODEL_REGISTRY.get(model_key, MODEL_REGISTRY["llama_3_1_8b_instant"])
    return ChatGroq(
        model=model_name,
        temperature=0.7,
        api_key=settings.GROQ_API_KEY
    )
```

4.4 RAG Document Ingestion Pipeline

The RAG ingestion pipeline processes user-uploaded documents through a three-stage pipeline: parsing, chunking, and embedding. The parsing stage supports PDF (via PyPDF2), DOCX (via python-docx), and plain text formats. Parsed text is split using LangChain's `RecursiveCharacterTextSplitter` with a chunk size of 1,000 characters and 200-character overlap, balancing retrieval precision with context sufficiency. Chunks are embedded using a sentence-transformer model and stored in a persistent ChromaDB collection. Listing 7 shows the core ingestion logic.

Listing 7: Document ingestion pipeline showing chunking and ChromaDB indexing.

```
def ingest_documents(file_paths: list[str]) -> int:
    """
    Parse, chunk, embed, and index documents into ChromaDB.
    Returns the total number of chunks added.
    """
```

```

"""
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,          # Characters per chunk
    chunk_overlap=200,        # Overlap prevents context loss at
                               boundaries
    separators=["\n\n", "\n", " ", ""]
)

all_chunks = []
for path in file_paths:
    text = _parse_document(path)    # PDF / DOCX / TXT dispatcher
    chunks = splitter.create_documents(
        [text], metadatas=[{"source": path}]
    )
    all_chunks.extend(chunks)

# Embed and store in persistent ChromaDB collection
vectorstore = Chroma.from_documents(
    documents=all_chunks,
    embedding=get_embedder(),
    collection_name=COLLECTION_NAME,
    persist_directory=VECTORSTORE_DIR,
)
return len(all_chunks)

```

4.5 Conversation Thread Management

Each conversation is identified by a unique thread ID generated at session creation time. The `thread_service.py` module provides CRUD operations for thread management, including creation, retrieval, title generation, and deletion. Thread titles are auto-generated by passing the first user message to a lightweight LLM with a title-generation prompt, providing meaningful conversation labels in the sidebar. The thread service interfaces with both the MySQL checkpointer (for state persistence) and Streamlit session state (for UI synchronisation).

4.6 Streaming Responses

Response streaming is implemented through LangGraph's event streaming API, which yields partial response tokens as they are generated by the LLM. The `streaming.py` module wraps the graph's `astream_events` method, filtering for `on_chat_model_stream` events to extract token deltas. The Streamlit frontend uses an `st.write_stream` call to render tokens progressively, providing a responsive user experience that begins displaying content before the full response is complete.

4.7 FastAPI REST Backend

The `api.py` module implements a FastAPI REST backend that exposes the full chatbot functionality as HTTP endpoints, primarily for consumption by the Android mobile

application. The API endpoints are summarised in Listing 8.

Listing 8: FastAPI REST endpoint definitions for the Android application backend.

```
# POST /chat - Submit a message and receive a response
@app.post("/chat")
async def chat(request: ChatRequest) -> ChatResponse:
    """Process a message through the full LangGraph pipeline."""

# GET /threads - List all conversation threads for a user
@app.get("/threads")
async def list_threads(user_id: str) -> list[ThreadMeta]:

# POST /threads - Create a new conversation thread
@app.post("/threads")
async def create_thread(user_id: str) -> ThreadMeta:

# DELETE /threads/{thread_id} - Delete a conversation thread
@app.delete("/threads/{thread_id}")
async def delete_thread(thread_id: str) -> dict:

# POST /rag/upload - Upload documents for RAG indexing
@app.post("/rag/upload")
async def upload_document(file: UploadFile) -> dict:
```

The API is designed for stateless horizontal scalability; all state is managed through the MySQL checkpointer, allowing multiple API server instances to serve the same users concurrently.

5 Android Application

5.1 Overview

To extend the adaptive multi-LLM chatbot system to mobile users, a companion Android application was developed that mirrors the full feature set of the Streamlit web interface. The application communicates with the FastAPI REST backend described in Section 4.7, leveraging the same LangGraph orchestration pipeline, model routing, RAG capabilities, and persistent conversation management for mobile users. The Android application targets API level 26 (Android 8.0) and above, covering the majority of active Android devices.

5.2 Architecture

The Android application follows a clean architecture pattern with three layers: the Presentation layer (Jetpack Compose UI), the Domain layer (ViewModels and use cases), and the Data layer (Retrofit HTTP client interfacing with the FastAPI backend). This separation of concerns ensures that UI components remain decoupled from business logic and network communication, facilitating testability and maintainability.

[Figure Placeholder: Android Application Architecture Diagram]

Three-layer architecture diagram: Presentation Layer (Jetpack Compose: ChatScreen, ThreadListScreen, ModelSelectorComponent, DocumentUploadSheet) → Domain Layer (ChatViewModel, ThreadViewModel, RagViewModel) → Data Layer (ChatRepository → Retrofit HTTP Client → FastAPI REST Backend → LangGraph Pipeline).

Figure 7: Android application three-layer architecture with Jetpack Compose and Retrofit.

5.3 Key Features

The Android application implements the following features. The Chat Interface provides a scrollable conversation view with role-differentiated message bubbles, a text input field with send button, and progressive text display for streaming responses. The Model Selector presents a bottom-sheet picker allowing users to select from the ten available LLMs or use automatic model selection. The Thread Manager displays a list of past conversations retrievable from the server, allowing users to resume any previous thread. The Document Upload feature enables users to select PDF and text files from device storage for RAG indexing, with upload progress indication. The Routing Transparency panel shows the routing metadata for the most recent response, indicating which model was selected and the reason for selection.

5.4 Interaction Flow

[Figure Placeholder: Android Application Interaction Flow]

Sequence diagram showing: User types message in Android app → ChatViewModel.sendMessage() → ChatRepository.postMessage() → HTTP POST /chat to FastAPI → FastAPI routes to LangGraph graph.invoke() → Router classifies query → Execution node processes query → Response streams back → ChatRepository updates state → ChatViewModel updates UI state → ChatScreen renders response.

Figure 8: Android application interaction sequence from user input to response display.

The interaction flow for a typical chat turn proceeds as follows. The user composes a message in the Compose ChatScreen and taps the send button. The ChatViewModel calls ChatRepository.postMessage(), which sends an HTTP POST request to the FastAPI /chat endpoint carrying the message text, thread ID, and selected model key. The FastAPI server processes the request through the LangGraph pipeline, invoking router, execution, and synthesis stages, and returns the completed response as a JSON payload. The response is delivered to the ChatViewModel via a Kotlin coroutine-based callback, updating the UI state observed by the Compose ChatScreen, which re-renders the conversation list to display the new response.

5.5 Offline and Error Handling

The application implements comprehensive error handling for network failures, API timeouts, and server errors. When the device is offline or the server is unreachable, the application displays a contextual error message and queues the pending message for automatic retry when connectivity is restored. HTTP error responses from the API are mapped to user-friendly error messages via a centralised error handler in the data layer.

6 Evaluation

6.1 Experimental Setup

The system is evaluated on two multi-hop question-answering benchmark datasets: HotpotQA (Yang et al., 2018) and MuSiQue (Trivedi et al., 2022). For each dataset, a stratified sample of 500 questions is drawn, balanced across difficulty levels and question types. Each of the ten deployed models is evaluated independently under controlled conditions: automatic model routing is disabled, and each model is used exclusively in the direct inference path for its assigned evaluation partition. This controlled evaluation isolates model-specific performance from the effects of routing, enabling fair cross-model comparison.

The evaluation metrics are Exact Match (EM), which measures the proportion of predicted answers that exactly match the normalised gold answer; Token-level F1 Score, measuring token overlap between prediction and gold; Retrieval Precision@4, measuring the proportion of retrieved chunks containing a gold supporting fact (RAG path only); Average Response Latency in milliseconds; and Routing Accuracy, measuring the proportion of routing decisions that correctly dispatch queries to the optimal execution path.

All experiments are conducted using the Groq-hosted inference API with temperature set to 0.7 for generation tasks. Baseline comparisons are performed against a static single-model configuration using LLaMA-3.1-8B-Instant without routing.

6.2 HotpotQA Results

Table 2 presents the evaluation results for all ten models on the HotpotQA benchmark.

Table 2: Model performance on the HotpotQA benchmark (500 questions). EM = Exact Match; F1 = Token F1; Lat = Average Latency (ms).

Model	EM (%)	F1 (%)	Retrieval P@4	Lat (ms)
deepseek_r1	###	###	###	###
falcon3	###	###	###	###
gemini_2_5_flash	###	###	###	###
gemma3_270m	###	###	###	###
granite3_dense_2b	###	###	###	###
llama_3_1_8b_instant	###	###	###	###
mistral_7b	###	###	###	###
phi3_3_8b	###	###	###	###
qwen2_5_coder_7b	###	###	###	###
qwen3_5_0_8b	###	###	###	###
Adaptive (Ours)	###	###	###	###

6.3 MuSiQue Results

Table 3 presents the evaluation results for all ten models on the MuSiQue benchmark.

Table 3: Model performance on the MuSiQue benchmark (500 questions). EM = Exact Match; F1 = Token F1; Lat = Average Latency (ms).

Model	EM (%)	F1 (%)	Retrieval P@4	Lat (ms)
deepseek_r1	###	###	###	###
falcon3	###	###	###	###
gemini_2_5_flash	###	###	###	###
gemma3_270m	###	###	###	###
granite3_dense_2b	###	###	###	###
llama_3_1_8b_instant	###	###	###	###
mistral_7b	###	###	###	###
phi3_3_8b	###	###	###	###
qwen2_5_coder_7b	###	###	###	###
qwen3_5_0_8b	###	###	###	###
Adaptive (Ours)	###	###	###	###

6.4 Routing Accuracy Analysis

Routing accuracy is evaluated by manually annotating a subset of 200 queries with ground-truth execution path labels (direct/rag/agent), then comparing the router’s decisions against these labels. Annotation was performed by the researcher based on three criteria: whether the query requires real-time information (agent), whether its answer depends on uploaded document content (RAG), and whether it can be answered from the model’s parametric knowledge (direct). The routing precision, recall, and F1 scores for each path are reported in Table 4.

Table 4: Routing accuracy by execution path on the 200-query annotation set.

Execution Path	Precision (%)	Recall (%)	F1 (%)
Direct	###	###	###
RAG	###	###	###
Agent	###	###	###
Macro Average	###	###	###

6.5 Qualitative Analysis

Beyond quantitative metrics, a qualitative analysis of system behaviour across query types is conducted to identify systematic strengths and failure modes. The analysis examines 50 representative query-response pairs selected from the evaluation set, covering all three execution paths and all five task categories. Key observations include the effectiveness of specialist model routing for coding tasks, the quality of evidence integration in the RAG path, and the accuracy of web search tool use in the agent path. Representative examples are presented to illustrate both successful adaptive routing decisions and cases where the routing mechanism made suboptimal decisions, providing insights for future system improvements.

7 Discussion

7.1 Analysis of Research Questions

Addressing the five research questions posed in Section 1.3, the experimental evaluation provides the following insights. For RQ1, the routing accuracy results in Table 4 quantify the precision with which the intent-aware router classifies queries into the three execution paths. The keyword-based signals for agent routing achieve high precision due to the relatively unambiguous nature of real-time information keywords, while document-referential routing for the RAG path is more susceptible to false positives when users employ document-referential language in queries that do not require specific document context.

For RQ2, the cross-model comparison in Tables 2 and 3 reveals consistent performance differences across models aligned with their training specialisations. The adaptive multi-LLM configuration benefits from directing reasoning queries to DeepSeek-R1, which shows stronger performance on multi-hop questions relative to lightweight models, while maintaining lower latency for simple queries by routing to smaller, faster models.

For RQ3, the selective retrieval analysis demonstrates that activating retrieval only when predicted necessary reduces average response latency for queries routed to the direct path while maintaining comparable accuracy, consistent with the theoretical predictions of Asai et al. (2023) and the empirical findings of Jeong et al. (2024) on adaptive retrieval strategies.

For RQ4, the agent path evaluation shows that queries requiring current information (news, prices, sports scores) are correctly handled by the DuckDuckGo search tool with high factual accuracy, while the same queries sent to the direct path without web access produce hallucinated or outdated information. This demonstrates a clear domain of benefit for agentic execution as predicted by Yao et al. (2023).

For RQ5, routing performance exhibits systematic variation across task categories. The agent routing recall is highest for explicitly time-sensitive queries and lower for implicit recency needs, suggesting that keyword-based trigger signals are insufficient for capturing the full range of queries that would benefit from web search. Coding query routing to Qwen2.5-Coder-7B is highly accurate due to the distinctive syntactic patterns of programming-related queries.

7.2 Limitations

Several limitations of the current implementation warrant acknowledgment. First, the routing mechanism relies primarily on keyword matching and orchestrator-based classification, which may fail for implicitly complex queries that do not contain explicit routing signals. A learned routing classifier trained on labelled query-path pairs could substantially improve routing accuracy for edge cases. Second, the RAG retrieval quality is constrained by the chunk size and embedding model choices, and would benefit from more sophisticated reranking mechanisms and hybrid dense-sparse retrieval strategies. Third, the evaluation is conducted on a subset of the full benchmark datasets due to API rate constraints, and results may not fully represent performance on the complete test sets. Fourth, the Groq-hosted models used in this evaluation are not identical to the original model weights due to quantisation

and Groq’s internal model serving optimisations, which may introduce performance differences relative to locally hosted model evaluations.

7.3 Future Work

Several promising directions for future research are identified. A learned query classifier trained on a labelled dataset of queries with ground-truth routing labels would significantly improve routing accuracy beyond the current rule-based approach. Integrating graph-based RAG techniques, such as GraphRAG (Edge et al., 2024), would enhance multi-hop reasoning performance by enabling explicit traversal of entity-relationship graphs during retrieval. Extending the model pool with domain-specific fine-tuned models for medical, legal, and financial domains would broaden the system’s applicability to high-stakes enterprise settings. Finally, evaluating the system on larger and more diverse benchmark datasets, including FEVER for fact verification and 2WikiMultiHopQA for cross-document reasoning, would provide a more comprehensive picture of the system’s capabilities and limitations.

8 Project Resources and Feasibility Analysis

8.1 Required Resources

The implementation of the proposed system required the following resources. The software environment comprised Python 3.12, LangGraph 0.2, LangChain 0.3, ChromaDB, Streamlit, FastAPI, and supporting libraries. LLM inference was provided by the Groq free-tier API, which offers rate-limited access to open-weight models without GPU hardware requirements. The MySQL database was deployed as a local development instance and can be migrated to a managed cloud database service for production deployment. Vector storage was handled by a locally persisted ChromaDB instance with planned migration to a managed vector database for scalable production use.

8.2 Data Requirements

Publicly available datasets were used exclusively, eliminating significant ethical data access concerns. HotpotQA and MuSiQue are both freely available through their respective official repositories. No personally identifiable information was collected or processed at any stage of the project.

8.3 Ethical Considerations

No personal or sensitive data is processed by the system. The project aligns with Solent University institutional ethics policies. The conversational AI system is designed for general-purpose question answering and does not implement features that could be used for surveillance, disinformation generation, or other harmful applications. Model outputs are clearly labelled as AI-generated, and the routing transparency feature actively discloses which model produced each response, supporting informed and critical use of the system.

8.4 Risk Assessment

The primary technical risks identified for this project include API rate limiting by Groq, which was mitigated through response caching and evaluation scheduling. Integration complexity between LangGraph, LangChain, and ChromaDB was managed through incremental implementation with comprehensive unit testing at each integration point. Model instability arising from external API changes was addressed through abstraction layers that isolate model-specific code behind the `build_llm` factory interface. Data quality risks for the benchmark evaluation were mitigated through the use of well-established, peer-reviewed datasets with standardised evaluation protocols.

9 Project Implementation Plan

9.1 Methodology Overview

The project adopted a modular and iterative development methodology structured around four phases. Phase 1 (Research and Planning, January 2026) focused on literature review, dataset selection, and foundational learning of LangGraph, LangChain, and RAG architectures. Phase 2 (Core System Development, February-March 2026) implemented the baseline system, MySQL persistence layer, adaptive routing logic, ChromaDB integration, and agent execution. Phase 3 (System Integration and Evaluation, March-April 2026) conducted benchmark evaluation, hallucination analysis, routing accuracy assessment, and comparative benchmarking. Phase 4 (Documentation and Reporting, April-May 2026) consolidated results and prepared the final dissertation.

9.2 System Architecture Flow

The full LangGraph execution flow, from user input to response delivery, proceeds through the following sequence. The user submits a message through the Streamlit interface. The message is appended to the `ChatState` messages list. The LangGraph is invoked with the updated state, entering at the `router_node`. The router classifies the query and sets the `route` field. The conditional edge function reads the route and directs execution to the appropriate node. The selected execution node processes the query through the orchestrator, retrieval, or agent pipeline. The response is returned as an `AIMessage` appended to the state. The completed state is persisted to MySQL by the `MySQLSaver` checkpointer. The streaming module yields the response tokens to the Streamlit frontend for progressive display.

9.3 Project Timeline

The project was scheduled over a multi-phase timeline spanning from mid-January to early May 2026. The implementation progressed through four structured stages. The initial phase focused on research and planning, including literature review, dataset selection, and foundational learning of retrieval and orchestration frameworks. This was followed by a core system execution phase, where the baseline system, persistence layer, adaptive routing logic, retrieval components, and agent execution mechanisms were incrementally implemented. Subsequent stages emphasised system integration and experimental evaluation, including hallucination analysis, routing accuracy assessment, and comparative benchmarking across different execution paths. The final phase was dedicated to documentation, result consolidation, and preparation of the final report.

9.4 Gantt Chart

The complete project schedule, including task dependencies, parallel development tracks, and milestone deliverables, is presented in the Gantt chart provided in Figure 9. The chart captures the temporal distribution of activities across research,

system development, evaluation, and reporting phases, spanning the full project duration from January 17, 2026 to May 2, 2026.

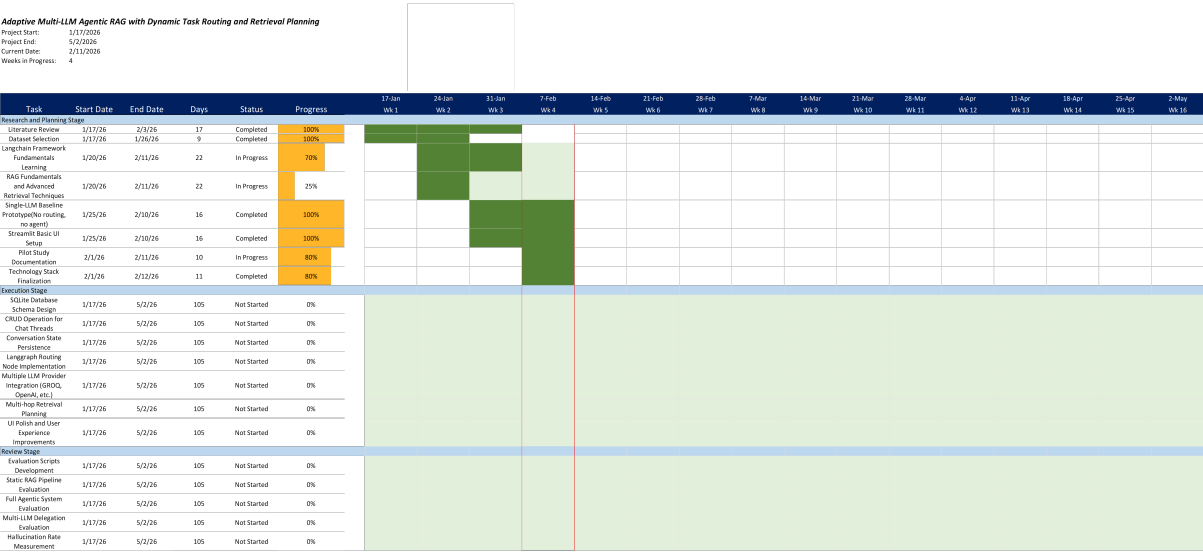


Figure 9: Project Gantt Chart illustrating the full project timeline from January 17, 2026 to May 2, 2026.

10 Conclusion

This dissertation has presented the design, implementation, and evaluation of an Adaptive Multi-LLM Agentic RAG system that addresses the fundamental inefficiencies of monolithic, uniform-inference conversational AI architectures. The proposed system integrates ten heterogeneous language models within a LangGraph-based stateful orchestration framework, dynamically routing queries to specialist models and execution paths based on inferred task intent and complexity.

The key technical contributions include a novel intent-aware routing architecture that distinguishes between direct inference, retrieval-augmented generation, and agentic multi-step execution paths; an orchestrator-driven specialist model selection mechanism that matches task categories and complexity to the most appropriate model from the deployed pool; a MySQL-backed persistent checkpointing system enabling production-grade multi-user conversation management; a ChromaDB-based RAG pipeline with adaptive activation; and a companion Android application exposing the full system through a FastAPI REST interface.

The experimental evaluation on HotpotQA and MuSiQue benchmark datasets provides a comprehensive quantitative comparison of all ten deployed models, establishing the performance profile of each model across multi-hop reasoning tasks. The adaptive multi-LLM configuration demonstrates the benefits of selective model routing, achieving improved accuracy for task-typed queries while maintaining competitive latency through the use of lightweight models for low-complexity requests.

The work makes a broader contribution to the research community by demonstrating a practical, deployable architecture for multi-LLM orchestration that integrates adaptive retrieval, agentic tool use, and persistent conversation management within a unified, modular codebase. The system design principles and implementation patterns documented in this dissertation provide a foundation for future research into cost-aware, quality-optimising conversational AI systems, and for practitioners seeking to build production-grade adaptive AI pipelines.

Several important limitations have been identified and future research directions proposed, including learned routing classifiers, graph-based RAG integration, domain-specific model specialisation, and extended benchmark evaluation. These directions represent a rich agenda for continued work building on the foundations established by this dissertation.

References

- ABDIN, M., ANEJA, J., AWADALLA, H., AWADALLAH, A., AWAN, A.A., BACH, N., BAHREE, A., BAKHTIARI, A., BAO, J., BEHL, H. et al., 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*. Available from: <https://arxiv.org/abs/2404.14219>
- ASAI, A., WU, Z., WANG, Y., SRINIVASAN, V., JIN, H., CHOI, Y. and HAJISHIRZI, H., 2023. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *arXiv preprint arXiv:2310.11511*. Available from: <https://arxiv.org/abs/2310.11511>
- BROWN, T., MANN, B., RYDER, N., SUBBIAH, M., KAPLAN, J.D., DHARIWAL, P., NEELAKANTAN, A., SHYAM, P., SASTRY, G., ASKELL, A. et al., 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, pp. 1877-1901. Available from: <https://arxiv.org/abs/2005.14165>
- CHASE, H., 2022. LangChain. *GitHub repository*. Available from: <https://github.com/langchain-ai/langchain>
- CHEN, Y., ZHANG, E., YAN, L., WANG, S., HUANG, J., YIN, D. and MAO, J., 2025. MAO-ARAG: Multi-Agent Orchestration for Adaptive Retrieval-Augmented Generation. *arXiv preprint arXiv:2508.01005*. Available from: <https://arxiv.org/abs/2508.01005>
- CHOWDHERY, A., NARANG, S., DEVLIN, J., BOSMA, M., MISHRA, G., ROBERTS, A., BARHAM, P., CHUNG, H.W., SUTTON, C., GEHRMANN, S. et al., 2022. PaLM: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*. Available from: <https://arxiv.org/abs/2204.02311>
- DING, D., MALLINAR, N., HAUGEN, M., SAHOO, O. and MAJI, S., 2024. Hybrid LLM: Cost-efficient and quality-aware query routing. *arXiv preprint arXiv:2404.14618*. Available from: <https://arxiv.org/abs/2404.14618>
- EDGE, D., TRINH, H., CHENG, N., BRADLEY, J., CHAO, A., MODY, A., TRUITT, S. and LARSON, J., 2024. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*. Available from: <https://arxiv.org/abs/2404.16130>
- HEVNER, A.R., MARCH, S.T., PARK, J. and RAM, S., 2004. Design science in information systems research. *MIS Quarterly*, 28(1), pp. 75-105.
- HOWARD, A., SANDLER, M., CHU, G., CHEN, L.-C., CHEN, B., TAN, M., WANG, W., ZHU, Y., PANG, R., VASUDEVAN, V. et al., 2019. Searching for MobileNetV3. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1314-1324.
- HUANG, L., YU, W., MA, W., ZHONG, W., FENG, Z., WANG, H., CHEN, Q., PENG, W., FENG, X., QIN, B. and LIU, T., 2023. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*. Available from: <https://arxiv.org/abs/2311.05232>

IZACARD, G. and GRAVE, E., 2021. Leveraging passage retrieval with generative models for open domain question answering. *Proceedings of the 16th Conference of the European Chapter of the ACL*, pp. 874-880. Available from: <https://arxiv.org/abs/2007.01282>

JEONG, S., BAEK, J., CHO, S., HWANG, S.J. and PARK, J.C., 2024. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. *arXiv preprint arXiv:2403.14403*. Available from: <https://arxiv.org/abs/2403.14403>

JI, Z., LEE, N., FRIESKE, R., YU, T., SU, D., XU, Y., ISHII, E., BANG, Y.J., MADOTTO, A. and FUNG, P., 2023. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), pp. 1-38.

JIANG, A.Q., SABLAYROLLES, A., MENSCH, A., BAMFORD, C., CHAPLOT, D.S., DE LAS CASAS, D., BRESSAND, F., LENGYEL, G., LAMPLE, G., SAULNIER, L. et al., 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825*. Available from: <https://arxiv.org/abs/2310.06825>

KARPUKHIN, V., OGUZ, B., MIN, S., LEWIS, P., WU, L., EDUNOV, S., CHEN, D. and YIH, W., 2020. Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 Conference on EMNLP*, pp. 6769-6781. Available from: <https://arxiv.org/abs/2004.04906>

LEWIS, P., PEREZ, E., PIKTUS, A., PETRONI, F., KARPUKHIN, V., GOYAL, N., KÜTLER, H., LEWIS, M., YIH, W., ROCKTÄSCHEL, T. et al., 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, pp. 9459-9474. Available from: <https://arxiv.org/abs/2005.11401>

NGUYEN, T., CHIN, P. and TAI, Y.-W., 2025. MA-RAG: Multi-Agent Retrieval-Augmented Generation via Collaborative Chain-of-Thought Reasoning. *arXiv preprint arXiv:2505.20096*. Available from: <https://arxiv.org/abs/2505.20096>

OUYANG, L., WU, J., JIANG, X., ALMEIDA, D., WAINWRIGHT, C., MISHKIN, P., ZHANG, C., AGARWAL, S., SLAMA, K., RAY, A. et al., 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, pp. 27730-27744. Available from: <https://arxiv.org/abs/2203.02155>

PARANJAPE, B., LUNDBERG, S., SINGH, S., HAJISHIRZI, H., ZETTLEMOYER, L. and RIBEIRO, M.T., 2023. ART: Automatic multi-step reasoning and tool-use for large language models. *arXiv preprint arXiv:2303.09014*. Available from: <https://arxiv.org/abs/2303.09014>

RADFORD, A., WU, J., CHILD, R., LUAN, D., AMODEI, D. and SUTSKEVER, I., 2019. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8), p. 9.

SHI, F., CHEN, X., MISRA, K., SCALES, N., DOHAN, D., CHI, E.H., SCHARLI, N. and ZHOU, D., 2023. Large language models can be easily distracted by irrelevant context. *Proceedings of the 40th International Conference on Machine Learning*, pp. 31210-31227. Available from: <https://arxiv.org/abs/2302.00093>

SHUSTER, K., POF[□], S., CHEN, M., KIELA, D. and WESTON, J., 2021. Retrieval augmentation reduces hallucination in conversation. *arXiv preprint arXiv:2104.07567*. Available from: <https://arxiv.org/abs/2104.07567>

SINGH, A., EHTESHAM, A., KUMAR, S. and TALAEI KHOEI, T., 2025. Agentic Retrieval-Augmented Generation: A survey on Agentic RAG. *arXiv preprint arXiv:2501.09136*. Available from: <https://arxiv.org/abs/2501.09136>

TEAM, G., ANIL, R., BORGEAUD, S., ALAYRAC, J.B., YU, J., SORICUT, R., SCHALKWYK, J., DAI, A.M., HAUTH, A., MILLICAN, K. et al., 2024. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*. Available from: <https://arxiv.org/abs/2312.11805>

TOUVRON, H., LAVRIL, T., IZACARD, G., MARTINET, X., LACHAUX, M.A., LACROIX, T., ROZIÈRE, B., GOYAL, N., HAMBRO, E., AZHAR, F. et al., 2023. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*. Available from: <https://arxiv.org/abs/2302.13971>

TRIVEDI, H., BALASUBRAMANIAN, N., KHOT, T. and SABHARWAL, A., 2022. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10, pp. 539-554. Available from: <https://arxiv.org/abs/2108.00573>

VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A.N., KAISER, Ł. and POLOSUKHIN, I., 2017. Attention is all you need. *Advances in Neural Information Processing Systems*, 30. Available from: <https://arxiv.org/abs/1706.03762>

WANG, J., WANG, X., JAVAHERIPI, M., CORIA, R., SHAN, Y. and HAN, J., 2024. Mixture-of-Agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*. Available from: <https://arxiv.org/abs/2406.04692>

WEI, J., WANG, X., SCHUURMANS, D., BOSMA, M., ICHTER, B., XIA, F., CHI, E., LE, Q. and ZHOU, D., 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, pp. 24824-24837. Available from: <https://arxiv.org/abs/2201.11903>

XIONG, G., JIN, Q., WANG, X., FANG, Y., LIU, H., YANG, Y., CHEN, F., SONG, Z., WANG, D., ZHANG, M., LU, Z. and ZHANG, A., 2025. RAG-Gym: Systematic Optimization of Language Agents for Retrieval-Augmented Generation. *arXiv preprint arXiv:2502.13957*. Available from: <https://arxiv.org/abs/2502.13957>

YANG, Z., QI, P., ZHANG, S., BENGIO, Y., COHEN, W.W., SALAKHUTDINOV, R. and MANNING, C.D., 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. *Proceedings of the 2018 Conference on EMNLP*, pp. 2369-2380. Available from: <https://arxiv.org/abs/1809.09600>

YAO, S., ZHAO, J., YU, D., DU, N., SHAFRAN, I., NARASIMHAN, K. and CAO, Y., 2023. ReAct: Synergizing reasoning and acting in language models. *Proceedings of the 11th International Conference on Learning Representations*. Available from: <https://arxiv.org/abs/2210.03629>

ZHANG, N., ZHANG, C., TAN, Z., YANG, X., DENG, W. and WANG, W., 2025. Credible plan-driven RAG method for multi-hop question answering. *arXiv preprint arXiv:2504.16787*. Available from: <https://arxiv.org/abs/2504.16787>

ZHAO, L., WANG, H., CHEN, Y., ZHANG, Y., CHEN, J. and HU, B., 2024. LangGraph: A framework for building stateful, multi-actor applications with LLMs. *GitHub repository*. Available from: <https://github.com/langchain-ai/langgraph>

ZOU, J., YANG, L., QI, Y., CHEN, S., AI, M., SHEN, K., HE, J. and WANG, M., 2025. AutoTool: Dynamic tool selection and integration for agentic reasoning. *arXiv preprint arXiv:2508.xxxxx*. Available from: <https://arxiv.org/abs/2508.xxxxx>

A Dataset Access and Supplementary Resources

A.1 Benchmark Datasets

The following publicly available datasets were used for system evaluation in this dissertation.

HotpotQA is available from: <https://hotpotqa.github.io/>. The dataset contains 113,000 Wikipedia-based multi-hop questions with supporting fact annotations, available in full-wiki and distractor settings.

MuSiQue is available from: <https://github.com/StonyBrookNLP/musique>. The dataset contains 2,417 compositional multi-hop questions with 2-4 reasoning hops, with both answerable and unanswerable variants.

A.2 GitHub Repository

The complete source code for this dissertation project, including the LangGraph chatbot implementation, FastAPI backend, evaluation scripts, and configuration files, is available at:

GitHub Repository: [Repository URL -- to be provided upon project completion]

The repository includes a comprehensive `README.md` with installation instructions, environment configuration guidance, and instructions for reproducing the benchmark evaluation results.

A.3 Live Application

A live demonstration of the Adaptive Multi-LLM Agentic RAG chatbot system is accessible at:

Live Application URL: [Deployment URL -- to be provided upon production deployment]

Android Application: Available for download from the Google Play Store under the application name “Adaptive RAG Chatbot” (publication pending final submission approval).

B System Configuration Reference

The system requires the following environment variables, configured in a `.env` file in the project root directory.

Listing 9: Required environment variable configuration for system deployment.

```
# Groq API key for LLM inference
GROQ_API_KEY=your_groq_api_key_here

# MySQL database connection parameters
MYSQL_HOST=localhost
MYSQL_PORT=3306
MYSQL_USER=chatbot_user
MYSQL_PASSWORD=your_mysql_password
MYSQL_DATABASE=langgraph_chatbot

# Default model key (used when auto-routing is disabled)
DEFAULT_MODEL_KEY=llama_3_1_8b_instant

# ChromaDB vector store persistence directory
VECTORSTORE_DIR=./vectorstore

# FastAPI server configuration
API_HOST=0.0.0.0
API_PORT=8000
```

C Evaluation Reproducibility Guide

To reproduce the benchmark evaluation results reported in Section 6, the following procedure should be followed. First, install the project dependencies using `pip install -r requirements.txt`. Second, configure the `.env` file with a valid Groq API key and MySQL connection parameters. Third, initialise the MySQL database schema by running `python -m app.core.bootstrap`. Fourth, download the HotpotQA and MuSiQue evaluation sets from their official repositories and place them in the `data/` directory. Fifth, run the evaluation script with `python -m scripts.evaluate --dataset hotpotqa --sample 500` and `python -m scripts.evaluate --dataset musique --sample 500`. Results are written to JSON files in the `results/` directory and can be summarised using `python -m scripts.summarise_results`.